# Subsystem Assignments

Team Organization and Responsibilities

NLC STEM Club - Electrical Systems Team

February 2026

# 1 SUBSYSTEM ASSIGNMENT & TRACKING DOCUMENT

- Fixed-Wing Environmental Monitoring UAV
- NLC STEM Club - Electrical Systems Team

# 2 SUBSYSTEM 1: POWER TEAM

# 3 TEAM MEMBERS

Owner: _____ Phone: _____ Email: _____
Assistant 1: _____ Phone: _____ Email: _____
Assistant 2: _____ Phone: _____ Email: _____

# 4 SUBSYSTEM SCOPE

You are responsible for ALL power generation, distribution, and safety for the aircraft. This includes battery management, voltage regulation, and ensuring stable power to all flight-critical and research payload systems.

# 5 MISSION-CRITICAL IMPORTANCE:

Without stable, properly distributed power, nothing else works. You are the foundation.

# 6 COMPONENTS ASSIGNED TO YOU

- OVONIC 4S LiPo Battery (14.8V, 2200mAh)
- B6 Battery Charger (80W)
- LiPo Safety Bag
- 40A ESC (includes 5V 3A BEC)
- External 5V UBEC (5-6A rating) [TO BE ORDERED]
- XT60 Connectors (Y-harness or splitter)
- Wire (12-14 AWG for high current, 18-20 AWG for BEC outputs)
- Heat shrink tubing (various sizes)
- Multimeter

WEEK 1 DELIVERABLES (Days 1-7)

- Battery Management Setup
- Charge battery to full (16.8V or 4.2V per cell) using B6 charger
- Verify balance: All 4 cells within 0.05V of each other

- Document battery voltage after full charge: _____V
- Label battery with charge date and voltage
- Create battery log sheet (date, voltage before use, voltage after use, flight time)
- ESC BEC Validation
- Connect battery to ESC (XT60)
- Measure ESC BEC output with multimeter
- Expected: 5.0V ± 0.1V Actual: _____V Status: PASS / FAIL
- Photograph ESC BEC output measurement
- External UBEC Setup (when arrives)
- Verify UBEC received and undamaged
- Solder input wires (14.8V from battery) - use 16-18 AWG wire
- Solder output wires (5V output) - use 18-20 AWG wire
- Apply heat shrink to all solder joints
- Connect UBEC input to battery (via test leads, not permanent yet)
- Measure UBEC output with multimeter
- Expected: 5.0V ± 0.1V Actual: _____V Status: PASS / FAIL
- Photograph UBEC and voltage measurement
- Power Distribution Planning
- Design XT60 Y-harness or splitter:
- Battery (+) → ESC (+) AND External UBEC (+)
- Battery (-) → ESC (-) AND External UBEC (-)
- Calculate wire lengths needed (measure from battery compartment to components)
- Sketch wiring layout on paper (including wire routing)
- Order/purchase any missing connectors or wire

WEEK 2 DELIVERABLES (Days 8-14)

- Power Distribution Assembly
- Build XT60 Y-harness (or install splitter)
- Solder all connections, apply heat shrink
- Label wires clearly: "Battery Main", "To ESC", "To UBEC"

- Test continuity with multimeter (0Ω from battery + to ESC + and UBEC +)
- Rail 1 (Flight-Critical) Assembly
- Create power distribution harness for Rail 1:
- ESC BEC (+5V) → FC, GPS, Receiver, Arduino
- Use 20-22 AWG wire, red for +5V, black for GND
- Create common ground bus (all grounds connected)
- Heat shrink all connections
- Label: "RAIL 1 - Flight-Critical"
- Rail 2 (Servo Power) Assembly
- Create power distribution harness for Rail 2:
- External UBEC (+5V) → All 4 servo red wires
- Use 18-20 AWG wire (servos draw more current)
- Create servo power bus (all 4 servo red wires to one +5V source)
- Create servo ground bus (all 4 servo brown wires to one GND)
- Heat shrink all connections
- Label: "RAIL 2 - Servo Power"
- Full System Power Test
- Connect full power system:
- Battery → Y-harness → ESC + UBEC
- ESC BEC → Rail 1
- External UBEC → Rail 2
- Power on system (no motor/servos connected yet)
- Measure voltages:
- Battery voltage: _____V (expect 14-16.8V)
- Rail 1 voltage: _____V (expect 5.0 ± 0.1V)
- Rail 2 voltage: _____V (expect 5.0 ± 0.1V)
- Verify common ground: Measure resistance from battery (-) to Rail 1 GND and
- Rail 2 GND. Should be 0Ω (continuity beep).
- Photograph entire power distribution setup

- Current Measurements (coordinate with Integration Team)
- Measure Rail 1 idle current (only FC + GPS + RX powered): _____mA
- Expected: ~320-400mA
- Measure Rail 1 with Arduino active: _____mA
- Expected: ~660mA
- Measure Rail 2 idle current (servos idle): _____mA
- Expected: ~40mA
- Measure Rail 2 with 2 servos moving: _____mA
- Expected: ~400-500mA
- Measure Rail 2 with all 4 servos moving: _____mA
- Expected: ~720-1200mA
- Voltage Sag Test (CRITICAL)
- Connect multimeter probes to Flight Controller 5V input (leave connected)
- Power system, note idle voltage: _____V
- Command all 4 servos to move simultaneously (full deflection)
- Note voltage during servo movement: _____V
- PASS CRITERIA: Voltage must stay ¿4.8V, ideally ¿4.9V
- If voltage drops below 4.8V → FAILURE → Increase BEC capacity or fix wiring
- Status: PASS / FAIL
- Documentation
- Create power system diagram with actual wire colors and lengths
- Take photos of all major connections
- Write 1-page summary: "Power System Integration Report"
- What you built
- Test results
- Issues encountered and how you solved them
- Lessons learned

# 7 SKILLS YOU WILL DEVELOP

- LiPo battery management and charging
- Soldering high-current connectors (XT60)
- Voltage regulation and power distribution
- Multimeter proficiency (voltage, current, continuity)
- Wire gauge selection for different current loads
- Heat shrink application for professional finish
- Systematic testing and validation procedures

# 8 POTENTIAL ISSUES & TROUBLESHOOTING

Problem: Battery won't charge or charger shows error

- → Check balance connector (all 5 pins connected)
- → Verify cell voltages (all cells ¿3.0V, ¡4.3V)
- → Ensure charger set to LiPo mode, 4S

Problem: BEC output voltage is wrong (not 5.0V)

- → Verify battery voltage is adequate (¿13.5V)
- → Check BEC solder connections (cold joint?)
- → Test BEC with minimal load (just multimeter, no components)
- → If still wrong, BEC may be defective

Problem: Voltage sag test fails (voltage drops below 4.8V)

- → Check wire gauge (may be too thin, causing voltage drop)
- → Check solder joints (high resistance = voltage drop)
- → Verify External UBEC is providing power to servos (not ESC BEC)
- → Measure current draw during test (may exceed BEC capacity)

# 9 NOTES & QUESTIONS

[Space for team to write notes during work]

# 10 SUBSYSTEM 2: FLIGHT ELECTRONICS TEAM

# 11 TEAM MEMBERS

Owner: _____ Phone: _____ Email: _____
Assistant 1: _____ Phone: _____ Email: _____
Assistant 2: _____ Phone: _____ Email: _____

# 12 SUBSYSTEM SCOPE

You are responsible for the BRAIN and CONTROL of the aircraft. This includes flight controller configuration, GPS navigation, radio control, and servo actuation.

# 13 MISSION-CRITICAL IMPORTANCE:

You control the aircraft's behavior. If you configure incorrectly, we crash. Precision and attention to detail are essential.

# 14 COMPONENTS ASSIGNED TO YOU

- MATEK F405-WMN Flight Controller
- BN-220 GPS Module (with compass)
- HAPPYMODEL ELRS EP2 Receiver
- Jumper Smart ELRS Transmitter (ground station)
- 4× RGBZONE ES08MA II Metal Gear Servos
- 40A ESC (for motor control signal)
- FLASH HOBBY D2830EVO Motor (1000KV)
- Folding Propeller (1060 or 1160) - DO NOT INSTALL YET
- Laptop with USB cable (for FC configuration)

WEEK 1 DELIVERABLES (Days 1-7)

- Flight Controller Setup
- Install flight controller configuration software on laptop:
- Option A: iNav Configurator (recommended for fixed-wing)
- Option B: ArduPilot Mission Planner
- Team Decision: We will use: _____
- Connect FC to laptop via USB

- Verify FC powers on (LED should light up)
- Connect to FC in configuration software
- Check current firmware version: _____
- Flash latest stable firmware if needed
- iNav: Version 7.0 or later
- ArduPilot: Plane version 4.5 or later
- Screenshot of FC connected in software
- GPS Module Setup
- Inspect GPS module and antenna
- Identify GPS connector type on FC (likely JST-SH 4-pin)
- Wire GPS to FC UART1:
- GPS VCC (red) → FC 5V or dedicated GPS 5V pad
- GPS TX (yellow) → FC RX1
- GPS RX (green) → FC TX1
- GPS GND (black) → FC GND
- In FC software, enable GPS on UART1
- Set GPS protocol: AUTO or UBX
- Set baud rate: 9600 or 115200 (try both if one doesn't work)
- Power FC via USB, take GPS module OUTDOORS
- Wait 5 minutes for GPS lock
- Verify GPS working:
- Satellite count: _____ (expect ¿8 for good lock)
- Latitude: _____ Longitude: _____
- GPS coordinates match your location? YES / NO
- Screenshot of GPS locked in software
- Receiver Binding & Configuration
- Read ELRS EP2 receiver binding instructions
- Power receiver (5V + GND from bench power supply or FC)
- Put receiver in bind mode (press bind button during power-up OR use WiFi method)

- Put transmitter in bind mode (follow Jumper TX instructions)
- Verify bind successful (receiver LED behavior changes)
- Wire receiver to FC:
- RX VCC (red) → FC 5V
- RX GND (black) → FC GND
- RX TX (white/yellow) → FC RX2 (or dedicated SBUS pad)
- In FC software, enable CRSF or SBUS protocol on UART2 (check which receiver uses)
- Set baud rate: 420000 (CRSF) or 100000 (SBUS)
- Verify receiver channels active:
- Move TX sticks, observe channel values in FC software
- Roll stick (right stick L/R): Channel 1 should move 1000-2000
- Pitch stick (right stick U/D): Channel 2 should move 1000-2000
- Throttle stick (left stick U/D): Channel 3 should move 1000-2000
- Yaw stick (left stick L/R): Channel 4 should move 1000-2000
- Screenshot of receiver channels responding

WEEK 2 DELIVERABLES (Days 8-14)

- ESC Configuration & Motor Test

WARNING CRITICAL SAFETY: Remove propeller for ALL indoor testing

- Verify propeller is REMOVED
- Wire ESC signal cable to FC Motor 1 output:
- ESC Signal (yellow/white) → FC M1 signal pad
- ESC Ground (brown/black) → FC GND
- ESC +5V (red) → DO NOT CONNECT (clip this wire)
- In FC software, configure Motor 1 output for motor
- Calibrate ESC (follow ESC calibration procedure):
- 1. Disconnect battery
- 2. Set TX throttle to MAXIMUM
- 3. Connect battery (ESC beeps)
- 4. Wait for confirmation beeps

- 5. Set TX throttle to MINIMUM
- 6. ESC beeps to confirm calibration complete
- Arm FC (throttle low, arm switch or stick command)
- SLOWLY increase throttle to 10-20%
- Motor should spin (NO PROP ATTACHED)
- Observe motor direction:
- Looking at motor from FRONT of aircraft, motor should spin CLOCKWISE
- If COUNTER-CLOCKWISE, swap any two motor wires (ESC to motor connection)
- Test throttle response through full range (0-100%)
- Disarm, disconnect battery
- Video of motor spin test (upload to shared drive)
- Servo Wiring & Configuration
- Identify servo outputs on FC: S1, S2, S3, S4 (or PWM1-4)
- Wire servos to FC (signal only, power comes from Rail 2):
- Servo 1 (Left Aileron): Signal (orange) → FC S1, Power from Rail 2
- Servo 2 (Right Aileron): Signal (orange) → FC S2, Power from Rail 2
- Servo 3 (Elevator): Signal (orange) → FC S3, Power from Rail 2
- Servo 4 (Rudder): Signal (orange) → FC S4, Power from Rail 2
- In FC software, configure servo outputs:
- Output 1 (S1): Type = Servo, Function = Aileron (or Left Aileron if separate)
- Output 2 (S2): Type = Servo, Function = Aileron (or Right Aileron if separate)
- Output 3 (S3): Type = Servo, Function = Elevator
- Output 4 (S4): Type = Servo, Function = Rudder
- For each servo, set:
- Min value: 1000µs (may adjust later)
- Max value: 2000µs (may adjust later)
- Center: 1500µs
- Rate: 100% (adjust in TX if needed)
- Servo Direction & Endpoint Configuration

- Power system (battery connected, FC + Servos powered)
- Verify all 4 servos center when FC armed
- Test each control input and verify CORRECT direction:
- RIGHT STICK LEFT (Roll Left - want left wing down, right wing up):
- Servo 1 (Left Aileron): Should move DOWN (trailing edge down)
- Servo 2 (Right Aileron): Should move UP (trailing edge up)
- If wrong direction, enable REVERSE in FC for that servo
- RIGHT STICK RIGHT (Roll Right - want right wing down, left wing up):
- Servo 1 (Left Aileron): Should move UP
- Servo 2 (Right Aileron): Should move DOWN
- If wrong direction, enable REVERSE in FC for that servo
- RIGHT STICK DOWN (Pitch Up - want nose to rise):
- Servo 3 (Elevator): Should move UP (trailing edge up)
- If wrong direction, enable REVERSE in FC for elevator
- RIGHT STICK UP (Pitch Down - want nose to lower):
- Servo 3 (Elevator): Should move DOWN (trailing edge down)
- If wrong direction, enable REVERSE in FC for elevator
- LEFT STICK LEFT (Yaw Left - want nose to turn left):
- Servo 4 (Rudder): Should move LEFT (trailing edge left)
- If wrong direction, enable REVERSE in FC for rudder
- LEFT STICK RIGHT (Yaw Right - want nose to turn right):
- Servo 4 (Rudder): Should move RIGHT (trailing edge right)
- If wrong direction, enable REVERSE in FC for rudder
- Document servo directions:
- Servo 1: NORMAL / REVERSED
- Servo 2: NORMAL / REVERSED
- Servo 3: NORMAL / REVERSED
- Servo 4: NORMAL / REVERSED

- Adjust servo endpoints if needed (limit travel to prevent binding):
- Manually move control surfaces to check for binding
- If binding at full deflection, reduce endpoints in FC (e.g., 1100-1900 instead of 1000-2000)
- Failsafe Configuration
- In FC software, configure failsafe:
- Trigger: Loss of radio signal (RX_LOSS)

### 14.1 Action

- Throttle → 0% (motor off)
- Servos → Hold last position OR Center (team decision)
- Recommended: Hold position (allows glide)
- Duration: Activate after 2 seconds of signal loss
- Test failsafe:
- 1. Power system, arm FC
- 2. Set throttle to 50%
- 3. Turn OFF transmitter
- 4. Verify motor cuts within 2 seconds
- 5. Verify servos hold position or center
- 6. Turn ON transmitter, verify control restored
- Failsafe Status: WORKING / NEEDS ADJUSTMENT
- Documentation
- Create wiring diagram showing FC connections (FC → GPS, RX, ESC, Servos)
- Screenshot of all FC configuration pages (save as PDF)
- Write 1-page summary: "Flight Electronics Integration Report"
- Firmware version and settings
- GPS, receiver, motor, servo configuration
- Issues encountered and solutions
- Lessons learned

# 15 SKILLS YOU WILL DEVELOP

- Flight controller firmware configuration (iNav or ArduPilot)
- GPS module interfacing and troubleshooting
- Radio control system binding and configuration (ELRS)
- Servo calibration and direction tuning
- ESC calibration and motor direction verification
- Failsafe programming for safe flight operations
- Systematic testing of control surfaces

# 16 POTENTIAL ISSUES & TROUBLESHOOTING

Problem: GPS won't lock (satellite count = 0)

- → Ensure testing OUTDOORS (GPS signals blocked indoors)
- → Check GPS wiring (TX/RX may be swapped)
- → Verify GPS enabled on correct UART in FC software
- → Try different baud rates (9600, 38400, 115200)
- → Wait longer (initial lock can take 5-10 minutes)

Problem: Receiver not binding to transmitter

- → Ensure RX and TX have same ELRS firmware version (update if mismatch)
- → Try WiFi binding method instead of button method (or vice versa)
- → Verify receiver is powered (5V + GND connected)

Problem: Receiver channels not showing in FC software

- → Check protocol setting (CRSF vs SBUS)
- → Verify baud rate matches protocol (420000 for CRSF, 100000 for SBUS)
- → Check wiring (RX TX → FC RX)
- → Try different UART port on FC

Problem: Motor won't spin or spins erratically

- → Verify ESC calibrated correctly
- → Check throttle channel mapping (usually Channel 3)
- → Ensure FC armed (check arming flags in software)

- → Verify motor output configured correctly in FC

Problem: Servo moving wrong direction

- → Enable REVERSE in FC for that servo channel
- → Double-check control surface should move in which direction

Problem: Servo jittering or twitching

- → Check power supply (voltage sag? Servos should be on Rail 2)
- → Reduce servo update rate in FC (try 50Hz instead of 200Hz)
- → Check for electrical noise (separate signal wires from motor wires)

## 17 NOTES & QUESTIONS

[Space for team to write notes during work]

## 18 SUBSYSTEM 3: RESEARCH PAYLOAD TEAM

## 19 TEAM MEMBERS

Owner: _____ Phone: _____ Email: _____
Assistant 1: _____ Phone: _____ Email: _____
Assistant 2: _____ Phone: _____ Email: _____

## 20 SUBSYSTEM SCOPE

You are responsible for the RESEARCH MISSION of the aircraft: collecting environmental data (temperature, humidity, pressure, air quality) and logging it to a MicroSD card.

## 21 MISSION-CRITICAL IMPORTANCE:

This is the PURPOSE of the entire project. Without functioning data logging, we have built a drone but accomplished no research. Your work defines success.

## 22 COMPONENTS ASSIGNED TO YOU

- Arduino Nano (ATmega328P)
- HiLetgo BME280 Sensor Module (Temperature, Humidity, Pressure)
- Ximimark MQ135 Air Quality Sensor (VOC, CO, NH)
- HiLetgo MicroSD Card Module (SPI interface)
- SanDisk 32GB MicroSD Card

- Taiss SPST Toggle Switch (2-pin, logging control)
- Laptop with Arduino IDE
- USB Mini cable (for Arduino programming)
- Jumper wires (Dupont M-F or F-F for prototyping)

WEEK 1 DELIVERABLES (Days 1-7)

- Arduino Setup
- Install Arduino IDE on laptop (version 1.8.19 or 2.x)
- Connect Arduino Nano via USB
- Install CH340 drivers if needed (for Nano clones)
- Select board: Tools → Board → Arduino Nano
- Select processor: Tools → Processor → ATmega328P (Old Bootloader if clone)
- Select port: Tools → Port → (select COM port with Nano)
- Upload "Blink" example sketch (File → Examples → 01.Basics → Blink)
- Verify LED blinks (confirms Arduino functional)
- BME280 Sensor Setup
- Install Adafruit BME280 library:
- Sketch → Include Library → Manage Libraries → Search "Adafruit BME280"
- Install "Adafruit BME280" library + dependencies
- Wire BME280 to Arduino:
- BME280 VCC → Arduino 5V or 3.3V (check if module is 5V tolerant)
- BME280 GND → Arduino GND
- BME280 SDA → Arduino A4 (SDA)
- BME280 SCL → Arduino A5 (SCL)
- Upload I2C Scanner sketch to find BME280 address:
- Expected address: 0x76 or 0x77
- Actual address found: 0x_____
- Upload BME280 test sketch (from Adafruit library examples)
- Open Serial Monitor (9600 baud)
- Verify readings:

- Temperature: _____°C (should be room temp, ˜20-25°C)
- Humidity: _____% (should be 30-70% indoors)
- Pressure: _____hPa (should be 980-1020 hPa depending on altitude)
- Test repeatability: Readings should be stable, not erratic
- Screenshot of Serial Monitor with BME280 readings
- MQ135 Sensor Setup
- Wire MQ135 to Arduino:
- MQ135 VCC → Arduino 5V (MQ135 requires 5V for heater)
- MQ135 GND → Arduino GND
- MQ135 AOUT (Analog Out) → Arduino A0
- (DOUT not used in this project)
- Upload simple analog read sketch:
- int gasValue = analogRead(A0);
- Serial.println(gasValue);
- Open Serial Monitor (9600 baud)
- Verify analog readings:
- Initial readings: _____ (0-1023 range)
- Note: MQ135 requires 24-48 hour burn-in for accurate readings
- During first test, readings will be unstable - this is normal
- Test sensitivity: Blow on sensor, value should change
- Document baseline value: _____ (this is "clean air" reference)
- MicroSD Card Module Setup
- Install SD library (included with Arduino IDE, no need to install separately)
- Format MicroSD card as FAT32 (use computer)
- Card size ¡32GB recommended (larger may have compatibility issues)
- Insert card into MicroSD module
- Wire MicroSD module to Arduino:
- SD VCC → Arduino 5V
- SD GND → Arduino GND

- SD MISO (DO) → Arduino D12 (MISO)
- SD MOSI (DI) → Arduino D11 (MOSI)
- SD SCK (CLK) → Arduino D13 (SCK)
- SD CS → Arduino D10 (or any digital pin, define in code)
- Upload SD CardInfo example sketch (File → Examples → SD → CardInfo)
- Open Serial Monitor (9600 baud)
- Verify SD card detected:
- Card type: _____ (SD, SDHC, or SDXC)
- Volume type: _____ (should be FAT16 or FAT32)
- Volume size: _____ MB
- Upload SD ReadWrite example (File → Examples → SD → ReadWrite)
- Verify file creation:
- Remove SD card, insert into computer
- Check for "test.txt" file
- Open file, verify text written correctly
- File creation Status: SUCCESS / FAILED
- Toggle Switch Setup
- Wire toggle switch to Arduino:
- Switch Pin 1 → Arduino D2
- Switch Pin 2 → Arduino GND
- Upload sketch to read switch state:
- pinMode(2, INPUT_PULLUP);
- int switchState = digitalRead(2);
- Serial.println(switchState);
- Open Serial Monitor
- Test switch:
- Switch OPEN: Serial Monitor shows 1 (HIGH)
- Switch CLOSED: Serial Monitor shows 0 (LOW)
- Switch Status: WORKING / NOT WORKING

WEEK 2 DELIVERABLES (Days 8-14)

- Integrated Data Logging Code Development
- Write Arduino sketch that combines all sensors + SD logging:

### 22.1 Required functionality

- 1. Initialize BME280, MQ135, SD card, toggle switch
- 2. Check if SD card is present and writable
- 3. Create CSV file with headers:
- "timestamp,temp_C,humidity_%,pressure_hPa,gas_level"
- 4. Read toggle switch state (D2)
- 5. If switch CLOSED (LOW):
- Read BME280 (temp, humidity, pressure)
- Read MQ135 (analog value from A0)
- Create timestamp (milliseconds since startup)
- Write CSV line to SD card
- Flush SD buffer (ensure data written immediately)
- Optional: Blink LED to indicate logging
- 6. If switch OPEN (HIGH):
- Do not log (or log at reduced rate)
- 7. Repeat every 1-5 seconds (team decision on logging rate)
- Code structure:
- Use functions for readability:
- void readBME280() { ... }
- void readMQ135() { ... }
- void logToSD() { ... }
- Add comments explaining each section
- Include error handling (SD card removed, sensor fail, etc.)
- Upload code to Arduino
- Open Serial Monitor to verify readings printing correctly
- Bench Testing & Validation
- Test 1: SD Card Detection
- Power Arduino with SD card inserted
- Verify Serial Monitor shows "SD card initialized"

- If fails, check wiring and card format
- Test 2: Sensor Data Validity
- Let MQ135 warm up for 5 minutes
- Verify BME280 readings reasonable:
- Temp: 15-30°C (room temp)
- Humidity: 20-80% (typical indoor range)
- Pressure: 980-1020 hPa (varies by altitude and weather)
- Verify MQ135 analog value: 0-1023 (any value in range is okay for now)
- Test 3: Toggle Switch Control
- Set switch to CLOSED (logging enabled)
- Verify Serial Monitor shows logging messages every 1-5 seconds
- Set switch to OPEN (logging disabled)
- Verify logging stops (or reduced rate)
- Set switch CLOSED again
- Verify logging resumes
- Test 4: CSV File Creation
- Run logging for 1 minute with switch CLOSED
- Power off Arduino
- Remove SD card, insert into computer
- Verify CSV file exists:
- File name: _____ (e.g., "datalog.csv" or timestamped name)
- File size: _____ bytes (should be ¿0)
- Open CSV file in Excel or text editor
- Verify columns:
- timestamp — temp_C — humidity_% — pressure_hPa — gas_level
- _____ — _____ — _____ — _____ — _____
- _____ — _____ — _____ — _____ — _____
- _____ — _____ — _____ — _____ — _____
- Check data is reasonable (no NaN, no -999 errors)

- File Status: VALID / ERRORS FOUND
- MQ135 Burn-In Procedure (CRITICAL)

WARNING WARNING: MQ135 requires 24-48 hours of continuous power before accurate readings

- Plan burn-in procedure:
- Option A: Power Arduino + MQ135 continuously for 48 hours before first flight
- Connect Arduino to USB power (not battery yet)
- Let run for 48 hours in ventilated area
- Option B: Power MQ135 separately from bench power supply
- 5V to VCC, GND to GND
- Leave powered for 48 hours
- Burn-in start date/time: _____
- Expected completion: _____
- During burn-in, monitor sensor readings every 6-12 hours:
- Hour 0: Gas value = _____
- Hour 6: Gas value = _____
- Hour 12: Gas value = _____
- Hour 24: Gas value = _____
- Hour 48: Gas value = _____ (should stabilize by now)
- Burn-in Status: COMPLETE / IN PROGRESS / NOT STARTED
- Arduino Power Integration
- Determine Arduino power source:
- Option A: Power from Rail 1 (Flight-Critical) via 5V + GND
- Option B: Power from Rail 2 (Servo Power) via 5V + GND
- Team Decision: Arduino powered from RAIL ___
- Disconnect Arduino from USB (will power from 5V rail now)
- Connect Arduino 5V pin to chosen rail
- Connect Arduino GND to common ground bus
- Power system (battery connected)
- Verify Arduino powers on (LED should light)

- Verify logging works (check SD card after test)
- Final Integration Test (5-Minute Flight Simulation)
- Full system powered (battery, FC, servos, Arduino)
- Insert fresh SD card (formatted, empty)
- Set toggle switch to CLOSED (logging enabled)
- Power on, let system run for 5 minutes
- Optional: Move sensors around, blow on MQ135, heat BME280 to create data variation
- Set toggle switch to OPEN (logging disabled)
- Power off
- Remove SD card, check CSV file:
- Number of data points: _____ (expect ~60-300 depending on logging rate)
- File size: _____ bytes
- Data looks reasonable? YES / NO
- Plot data in Excel (time vs temp, time vs humidity, time vs gas level)
- Screenshot of data plot
- Documentation
- Upload Arduino code to shared drive (with comments!)
- Create wiring diagram for Arduino + sensors
- Write CSV file format specification (column definitions)
- Write 1-page summary: "Research Payload Integration Report"
- Sensor specifications and ranges
- Data logging format
- Calibration procedure (if any)
- Known limitations (MQ135 not quantitatively calibrated, etc.)
- Lessons learned

# 23 SKILLS YOU WILL DEVELOP

- Arduino programming (C/C++)
- I2C communication protocol (BME280)
- Analog-to-digital conversion (ADC) reading (MQ135)
- SPI communication protocol (MicroSD)
- File I/O and CSV formatting
- Sensor interfacing and troubleshooting
- Data validation and quality control

# 24 POTENTIAL ISSUES & TROUBLESHOOTING

Problem: BME280 not detected (I2C scan shows no device)

- → Check wiring (SDA → A4, SCL → A5)
- → Verify power (VCC → 5V or 3.3V, check module specs)
- → Try both I2C addresses (0x76 and 0x77)
- → Ensure I2C pullup resistors present (usually on breakout module)

Problem: MQ135 readings always 0 or always 1023

- → Check wiring (AOUT → A0)
- → Verify 5V power (MQ135 heater requires 5V)
- → Sensor may be defective (test with multimeter: should read 0-5V on AOUT)

Problem: SD card not detected

- → Verify card formatted as FAT32 (not exFAT or NTFS)
- → Try different SD card (¡32GB recommended)
- → Check wiring (MISO, MOSI, SCK, CS pins)
- → Ensure VCC connected to 5V (some modules have onboard regulator)
- → Try different CS pin (change in code: const int chipSelect = 10;)

Problem: CSV file empty or corrupted

- → Add SD flush after each write: file.flush();
- → Ensure file.close() called before power off
- → Check SD card not full

- → Verify file opened successfully: if (!file) { Serial.println("Error"); }

  Problem: Data looks unrealistic (e.g., temp = -999)

- → Sensor not initializing properly (check begin() function return value)
- → Check sensor wiring and power
- → Add error handling in code:
- if (isnan(temp)) { temp = -999; } // Flag as error

## 25 NOTES & QUESTIONS

[Space for team to write notes during work]

## 26 SUBSYSTEM 4: INTEGRATION & TESTING TEAM

## 27 TEAM MEMBERS

Owner: _____ Phone: _____ Email: _____
Assistant 1: _____ Phone: _____ Email: _____
Assistant 2: _____ Phone: _____ Email: _____

## 28 SUBSYSTEM SCOPE

You are responsible for SYSTEMS INTEGRATION and VALIDATION. Your job is to ensure all subsystems work together, identify issues before flight, and verify the aircraft is electrically ready for operation.

## 29 MISSION-CRITICAL IMPORTANCE:

You are the last line of defense before flight. If you miss a faulty connection or electrical issue, we risk losing the aircraft. Attention to detail is everything.

## 30 COMPONENTS ASSIGNED TO YOU

- Multimeter (voltage, current, continuity, resistance)
- Clamp meter (for measuring high currents) [if available]
- Inline current sensor (if available)
- Wire strippers, cutters, crimpers
- Zip ties, foam tape, velcro straps
- Heat shrink tubing (various sizes)
- Electrical tape

- Labels / label maker
- Notebook for recording measurements

WEEK 1 DELIVERABLES (Days 1-7)

- Multimeter Training & Practice
- Learn multimeter functions:
- DC Voltage (DCV): For measuring battery, BEC outputs (0-20V range)
- DC Current (DCA): For measuring current draw (200mA, 2A, 10A ranges)
- Continuity (): For checking ground connections (beeps when 0Ω)
- Resistance (Ω): For checking short circuits or open connections
- Practice measurements on bench:
- Test 1: Measure AA battery voltage (expect ˜1.5V)
- Measured: _____V
- Test 2: Measure 9V battery voltage (expect ˜9V)
- Measured: _____V
- Test 3: Check continuity of wire (should beep)
- Status: PASS / FAIL
- Test 4: Measure resistance of wire (expect ¡1Ω)
- Measured: _____Ω
- Create multimeter quick reference guide (printout for team):
- How to measure voltage (red to +, black to -)
- How to measure current (in series with circuit)
- How to check continuity (across connection)
- Safety: Never measure current in voltage mode (blows fuse!)
- Component Inventory & Condition Check
- Work with all subsystem teams to verify component condition
- For each major component, check:
- Physical damage (cracks, bent pins, damaged wires)
- Connector condition (secure, not loose or damaged)
- Wire insulation (no exposed copper, no fraying)

- Document issues found:
- Component: _____ Issue: _____ Resolution: _____
- Component: _____ Issue: _____ Resolution: _____
- Common Ground Topology Planning
- Study system architecture (refer to Block Diagram document)
- Identify all ground connections needed:
- Battery negative (master ground)
- ESC ground
- External UBEC ground
- Flight Controller ground
- Arduino ground
- Servo grounds (4×)
- Sensor grounds (BME280, MQ135, MicroSD)
- Create ground wiring plan:

### 30.1 Use STAR TOPOLOGY from battery negative

- Battery (-) → Main GND bus → ESC, UBEC, FC, Arduino, Servos, Sensors
- Calculate total wire length needed for ground bus: _____ meters
- Recommended wire gauge for ground: 18-20 AWG (thick enough for current return)
- Wire Routing & Management Planning
- Study airframe layout (coordinate with Mechanical team)
- Plan wire routing:
- High-current wires (motor, battery): Shortest path, away from sensitive electronics
- Signal wires (servo signals, GPS, RX): Away from motor wires (noise reduction)
- Sensor wires: Away from motor/ESC, accessible for troubleshooting
- Create wire routing diagram (sketch on paper or CAD):
- Mark wire paths in airframe
- Identify zip tie attachment points
- Plan for strain relief at connectors

- Allow slack for vibration (don't pull wires tight)
- Estimate wire lengths needed:
- Battery to ESC: _____ cm
- Battery to UBEC: _____ cm
- ESC to Motor: _____ cm
- ESC BEC to FC: _____ cm
- UBEC to Servo Rail: _____ cm
- FC to GPS: _____ cm
- FC to RX: _____ cm
- FC to Servos (4×): _____ cm each
- Arduino to Sensors: _____ cm

WEEK 2 DELIVERABLES (Days 8-14)

- Continuity Testing (CRITICAL - Do this before first power-on)
- Set multimeter to continuity mode ( symbol)
- Verify all grounds connected (should beep for 0Ω connections):
- Test 1: Battery (-) to ESC ground
- Measured: _____Ω Status: PASS (¡1Ω) / FAIL
- Test 2: Battery (-) to External UBEC ground
- Measured: _____Ω Status: PASS / FAIL
- Test 3: Battery (-) to Flight Controller GND pad
- Measured: _____Ω Status: PASS / FAIL
- Test 4: Battery (-) to Arduino GND pin
- Measured: _____Ω Status: PASS / FAIL
- Test 5: Battery (-) to Servo 1 brown wire (ground)
- Measured: _____Ω Status: PASS / FAIL
- Test 6: Battery (-) to BME280 GND
- Measured: _____Ω Status: PASS / FAIL
- If ANY test fails (no beep, high resistance):

- → STOP and trace wiring
- → Fix ground connection before proceeding
- → Re-test until all grounds show continuity
- Voltage Measurements (Validate Power Distribution)
- Power system with battery (no motor/servos connected initially)
- Measure voltages at key points:

### 30.2 Battery voltage (XT60 terminals)

- Expected: 14.0-16.8V Measured: _____V Status: PASS / FAIL

### 30.3 Rail 1 voltage (ESC BEC output, measure at FC 5V pad)

- Expected: 5.0 ± 0.1V Measured: _____V Status: PASS / FAIL

### 30.4 Rail 2 voltage (External UBEC output, measure at servo power rail)

- Expected: 5.0 ± 0.1V Measured: _____V Status: PASS / FAIL
- GPS module VCC (at GPS connector):
- Expected: 5.0 ± 0.1V Measured: _____V Status: PASS / FAIL

### 30.5 Receiver VCC

- Expected: 5.0 ± 0.1V Measured: _____V Status: PASS / FAIL

### 30.6 Arduino 5V pin

- Expected: 5.0 ± 0.1V Measured: _____V Status: PASS / FAIL
- If any voltage out of range:
- → Check connections (cold solder joint? Loose connector?)
- → Measure voltage at BEC output vs at component (voltage drop indicates bad connection)
- Current Measurements (Validate Power Budget)
- Measure current draw for each subsystem:
- Method: Use clamp meter (around wire) or inline current sensor

### 30.7 Rail 1 idle (only FC + GPS + RX powered, no Arduino yet)

- Expected: ~320-400mA Measured: _____mA Status: PASS / FAIL

### 30.8 Rail 1 with Arduino active (logging)

- Expected: ~660mA Measured: _____mA Status: PASS / FAIL

### 30.9 Rail 2 idle (servos idle, no movement)

- Expected: ~40mA Measured: _____mA Status: PASS / FAIL

### 30.10 Rail 2 with 2 servos moving

- Expected: ~400-500mA Measured: _____mA Status: PASS / FAIL

### 30.11 Rail 2 with all 4 servos moving

- Expected: ~720-1200mA Measured: _____mA Status: PASS / FAIL

### 30.12 Battery total current (motor at 50% throttle, NO PROP)

- Expected: ~10-12A Measured: _____A Status: PASS / FAIL
- Compare measured currents to Power Budget document predictions
- If currents significantly higher than expected (¿20% over):
- → Investigate (component drawing excessive current? Short circuit?)
- Voltage Sag Test (MOST CRITICAL TEST)

WARNING This test verifies FC won't brown out during servo load

- Setup:
- Connect multimeter probes to Flight Controller 5V input (leave connected)
- Power full system (battery connected, all subsystems active)
- Set multimeter to DC voltage, 20V range
- Baseline voltage (idle, no servo movement):
- Measured: _____V (should be ~5.0V)
- Voltage under load (move all 4 servos simultaneously, full deflection):
- Measured: _____V (MUST stay ¿4.8V, ideally ¿4.9V)
- PASS CRITERIA: Voltage must stay ¿4.8V during aggressive servo movement
- Status: PASS / FAIL
- If FAIL (voltage drops below 4.8V):

### 30.13 Possible causes

- Servos powered from Rail 1 instead of Rail 2 (check wiring!)
- Excessive current draw (measure with clamp meter)
- Insufficient BEC capacity (need larger UBEC?)
- High-resistance connection (voltage drop in wiring)
- Resolution: _____
- GPS Lock Test with Motor Running (Electrical Noise Test)

WARNING This test verifies motor/ESC noise doesn't interfere with GPS

- Setup:
- Power full system
- Wait for GPS lock (satellite count ¿8)
- Open FC configuration software, monitor GPS satellite count
- Baseline GPS lock (motor off):
- Satellite count: _____ (should be ¿8)
- GPS lock with motor running (50% throttle, NO PROP, outdoors):
- Satellite count: _____ (should NOT drop significantly)
- PASS CRITERIA: GPS maintains lock, satellite count drops by ¡3
- Status: PASS / FAIL
- If FAIL (GPS loses lock or satellite count drops significantly):

### 30.14 Possible causes

- Motor wires too close to GPS antenna
- Motor wires not twisted (increases EMI)
- GPS not shielded from motor/ESC

### 30.15 Mitigation

- Twist motor wires together (reduces electromagnetic interference)
- Increase separation between motor/ESC and GPS (¿10cm minimum)
- Add ferrite bead to motor wires (reduces high-frequency noise)
- Shield GPS module with aluminum foil or copper tape (if desperate)

- Resolution: _____
- Wire Management & Vibration Isolation
- Secure all wires with zip ties (every 10-15cm)
- Ensure no wires rub against:
- Control surfaces (ailerons, elevator, rudder)
- Sharp edges (can cut through insulation)
- Moving parts (servo arms, linkages)
- Add strain relief at connectors:
- Loop wire before connector, secure with zip tie
- Prevents connector pull-out during vibration
- Mount vibration-sensitive components on foam:
- Flight Controller (double-sided foam tape)
- GPS module (foam pad)
- Arduino + sensors (foam pad or rubber standoffs)
- Label all major wires:
- Battery + / Battery -
- Rail 1 +5V / Rail 1 GND
- Rail 2 +5V / Rail 2 GND
- Motor wires (Phase A, B, C)
- Servo signals (S1, S2, S3, S4)
- Pre-Flight Electrical Checklist Creation
- Create laminated checklist for pre-flight verification:

# 31 PRE-FLIGHT ELECTRICAL CHECKLIST:

- Battery voltage ¿14.0V (measure with multimeter)
- Battery balance: All cells within 0.05V (check via balance connector)
- All connectors secure (XT60, servo connectors, FC connections)
- No loose wires or exposed metal
- Propeller secure (spinner nut tight) [only when ready for flight]

- GPS module LED blinking (indicates searching for lock)
- FC status LED normal pattern (not rapid blinking = error)
- Receiver LED solid (indicates bound to TX)
- Arduino logging active (if switch ON, LED indicates activity)
- Battery charged within last 24 hours (LiPo self-discharge)
- Fire extinguisher and first aid kit present
- LiPo safety bag available (for post-flight storage)
- Print 5 copies of checklist (use for each flight/test)
- Full System Integration Test

WARNING This is the final validation before airframe installation

- Setup: All electrical systems on bench, fully wired, battery connected
- Power on sequence:
- 1. Verify all connections secure
- 2. Turn ON transmitter (throttle at zero)
- 3. Connect battery
- 4. Wait 30 seconds for system initialization
- 5. Verify GPS lock (wait 2-5 minutes outdoors if needed)
- Functional tests:
- Flight Controller status LED: Normal pattern? YES / NO
- GPS lock: Satellite count ¿8? _____ satellites
- Receiver control: Move TX sticks, servos respond? YES / NO
- Motor response: Arm FC, increase throttle (NO PROP), motor spins? YES / NO
- Servo directions: All 4 servos move correct direction? YES / NO
- Failsafe: Turn OFF TX, motor cuts within 2 sec? YES / NO
- Arduino logging: Toggle switch ON, data logging to SD card? YES / NO
- Power measurements under full load:
- Battery voltage (under load): _____V (should stay ¿14.0V)
- Battery current draw: _____A (motor at 50% throttle)
- Flight time estimate: Battery capacity (2.2Ah) / Avg current (A) = _____ hours

- (Multiply by 60 for minutes, expect ˜9-13 minutes)
- Integration Status: READY FOR AIRFRAME INSTALL / ISSUES FOUND
- If issues found, document and resolve before proceeding:
- Issue 1: _____
- Issue 2: _____
- Documentation
- Take photos of final wiring setup (all angles)
- Create "as-built" wiring diagram (showing actual wire colors and routing)
- Record all voltage and current measurements in spreadsheet
- Write 2-page summary: "System Integration Test Report"
- Test procedures and results
- Pass/fail status for each test
- Issues found and resolutions
- Readiness assessment for airframe integration
- Recommendations for flight testing

## 32 SKILLS YOU WILL DEVELOP

- Multimeter proficiency (voltage, current, continuity, resistance)
- System-level thinking (understanding interactions between subsystems)
- Electrical troubleshooting methodology
- Wire management and professional routing
- Vibration isolation for electronics
- Test procedure development and execution
- Documentation and reporting

## 33 POTENTIAL ISSUES & TROUBLESHOOTING

Problem: No continuity between grounds (multimeter doesn't beep)

- → Trace ground wiring, find break or missing connection
- → Check solder joints (cold joint may have high resistance)
- → Verify all components share common ground bus

Problem: Voltage correct at BEC output but low at component

- → Voltage drop indicates high resistance in wiring
- → Check wire gauge (may be too thin for current draw)
- → Check connectors (loose or dirty contacts)
- → Improve connection (thicker wire, better solder joint)

Problem: Current draw much higher than predicted

- → Check for short circuit (measure resistance, should be high when powered off)
- → Identify which component drawing excessive current (disconnect one at a time)
- → Component may be defective

Problem: Voltage sag test fails (FC voltage drops below 4.8V)

- → CRITICAL ISSUE - Do not fly!
- → Verify servos powered from Rail 2 (not Rail 1)
- → Check External UBEC output capacity (may need higher current rating)
- → Reduce servo load (limit travel in FC to reduce peak current)

Problem: GPS loses lock during motor test

- → Electrical noise from motor/ESC interfering with GPS
- → Twist motor wires together (reduces EMI)
- → Increase physical separation (¿10cm)
- → Add ferrite bead to motor power wires
- → Shield GPS module (aluminum foil, copper tape)

# 34 NOTES & QUESTIONS

[Space for team to write notes during work]

# 35 FINAL REMINDERS FOR ALL TEAMS

- Safety first, always. If you're unsure, ask.
- Document everything. Photos, measurements, notes. This is your portfolio.
- Test early and often. Don't wait until integration to discover issues.
- Communicate with other teams. Your subsystems must work together.

- Ask questions. No question is stupid. Confusion now prevents crashes later.
- Meet deadlines. Other teams depend on your deliverables.
- Take pride in your work. This is aerospace engineering, not a hobby project.

You have the knowledge, the tools, and the team. Now execute.
Build something great.